Дообучение больших языковых моделей

Обучение на инструкциях, PEFT и подготовка данных

Илья Новицкий

28 ноября 2026 · Блок 4 · Лекция 1

Содержание лекции

- Разберём, чем дообучение отличается от промпта и поиска по документам и когда оно оправдано.
- Посмотрим на обучение с учителем: цель, маскирование потерь и роль токена завершения.
- Обсудим форматы данных инструкций и шаблоны диалога конкретной модели.
- Сравним полное дообучение с LoRA и QLoRA и соберём процесс в Hugging Face TRL и PEFT.

Три способа изменить поведение модели

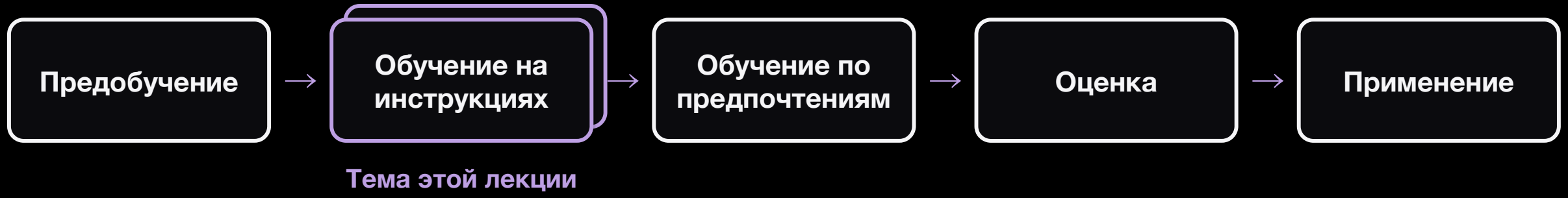
Способ	Что меняется	Когда изменение действует
Промпт и примеры	Текст на входе	Только в текущем запросе
Поиск по документам	Контекст из внешней базы	Пока база доступна приложению
Дообучение	Параметры модели	Во всех запросах к этой версии

- Способы сочетаются: дообученная модель тоже может получать документы и инструкции.
- Изменение весов требует отдельной версии модели, проверки и хранения.

Что такое дообучение

- Продолжение обучения уже предобученной модели на данных, отражающих нужную задачу.
- Модель сохраняет общие языковые закономерности и подстраивает поведение под новые примеры.
- Изменяются все параметры или их небольшая дополнительная часть — в зависимости от метода.
- Результат — новая версия модели, которую сравнивают с исходной на одинаковой проверке.

Этапы подготовки модели



- Предобучение даёт общие языковые закономерности; это самый дорогой этап.
- Сегодня — обучение на инструкциях и его эффективные варианты; предпочтения — на следующей лекции.
- Мы начинаем не с нуля: исходная модель обычно уже прошла несколько этапов.

Когда дообучение помогает

- Нужен устойчивый формат ответа, который промпт выдерживает не во всех случаях.
- Задача узкая и повторяющаяся, а размеченных примеров достаточно для обучения.
- Нужно сократить длинный промпт с инструкциями и примерами, чтобы уменьшить стоимость и задержку.
- Небольшая модель после дообучения может заменить большую на конкретной задаче.

Когда дообучение не нужно

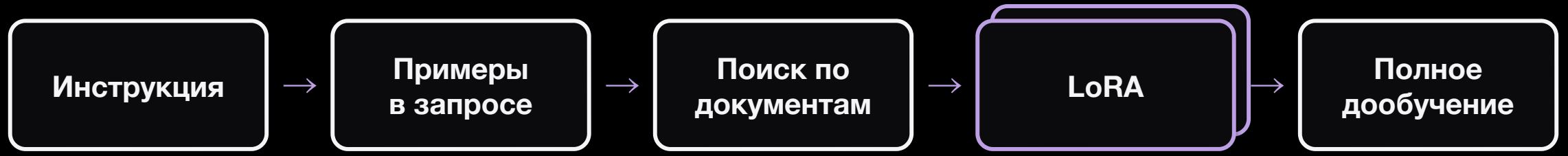
- Задачу удаётся решить инструкцией и несколькими примерами с приемлемым качеством.
- Требуются актуальные или часто меняющиеся сведения: их проще передавать через поиск.
- Нет данных для проверки: без отложенной выборки нельзя понять, стало ли лучше.
- Ошибки вызваны неясной постановкой задачи, а не ограничениями модели.

Выбор первого шага

Ситуация	С чего начать
Модель не понимает задачу	Уточнить инструкцию и добавить примеры
Не хватает сведений о предмете	Поиск по документам
Формат или стиль нестабилен	Примеры, затем дообучение
Длинный промпт дорог в применении	Дообучение небольшой модели
Нужно новое знание в весах	Осторожно: проверка на забывание

- Решение принимают по результатам на своей проверочной выборке, а не по общему впечатлению.

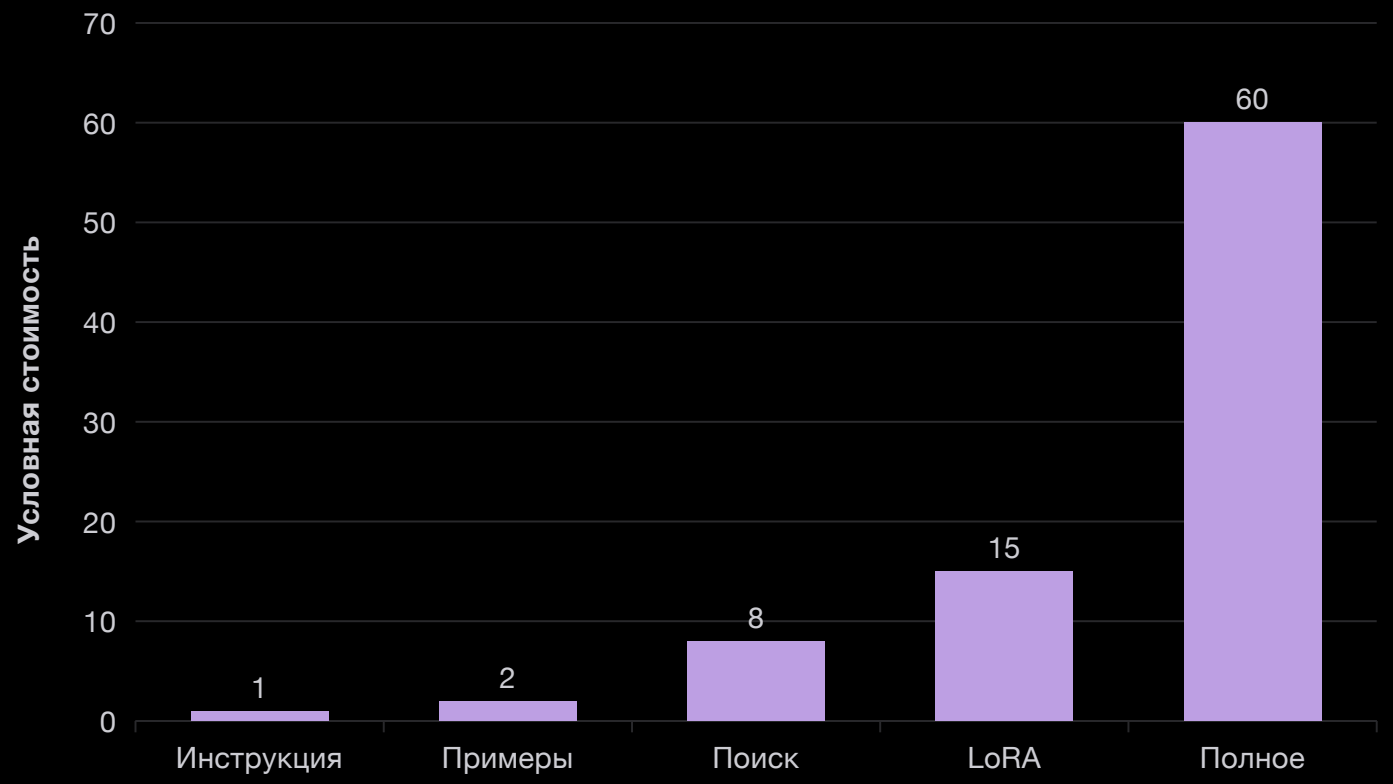
Порядок попыток



Чаще всего достаточно адаптера

- Каждый следующий шаг обычно дороже по данным, вычислениям и сопровождению.
- Предыдущие шаги дают базовую линию: дообучение сравнивают именно с ней.
- Порядок ориентировочный: при большом объёме данных шаги можно пропускать.

Условная стоимость эксперимента



- Учитываются подготовка данных, вычисления и проверка.
- Полное дообучение требует больше памяти и хранения отдельной копии весов.
- Числа схематичны и показывают порядок, а не результат измерения.

Обучение с учителем на инструкциях

- Обучающий пример состоит из входа — инструкции и контекста — и желаемого ответа.
- Модель учится продолжать вход этим ответом; цель та же, что при предобучении: следующий токен.
- Отличаются данные: вместо произвольного текста — диалоги и ответы в нужной форме.
- В англоязычной литературе этап называют supervised fine-tuning, сокращённо SFT.

Функция потерь на ответе

$$L = -(1/|y|) \sum_t \log P\theta(y_t \mid x, y_{<t})$$

- x — инструкция и контекст, y — эталонный ответ, θ — обучаемые параметры.
- Суммирование идёт по токенам ответа: модель штрафуют за низкую вероятность эталона.
- Токены входа участвуют в вычислениях как контекст, но обычно не входят в потери.
- Эталон задаёт один допустимый ответ; другие верные формулировки тоже получают штраф.

Какие токены участвуют в потерях

Часть примера	Видна модели	Входит в потери
Системная инструкция	Да	Обычно нет
Сообщение пользователя	Да	Обычно нет
Ответ ассистента	Да	Да
Токен конца сообщения	Да	Да
Выравнивание пакета (padding)	Нет	Нет

- Маска потерь задаёт, за что модель получает штраф, но не скрывает вход от внимания.
- Обучение только на ответах снижает риск, что модель начнёт генерировать вопросы.

Токен завершения ответа

- Модель должна научиться не только начинать ответ, но и останавливаться в нужный момент.
- Если в обучающем примере нет токена конца, после дообучения ответ может продолжаться бесконечно.
- Токен конца берут из шаблона модели; у разных семейств он различается.
- Проверка: сгенерировать ответ и посмотреть причину остановки — служебный токен или лимит.

Чему модель учится при дообучении

- Хорошо переносятся формат, стиль, длина ответа и правила выбора из заданного набора вариантов.
- Модель лучше следует узкой задаче, которая повторяется в обучающих примерах.
- Новые факты из небольшого набора усваиваются ненадёжно и могут провоцировать выдумки.
- Для знаний, которые часто меняются, полезнее поиск по документам с указанием источника.

Поведение и знания

Переносится надёжно

- Формат: JSON, таблица, метка класса
- Тон и длина ответа
- Правило выбора между похожими классами
- Отказ на запросы вне задачи

Переносится плохо

- Свежие факты и цифры
- Сведения, встречающиеся в одном примере
- Источник утверждения
- Знание, противоречащее предобучению

- Разделение условное: оно описывает частые наблюдения, а не строгое правило.

Сколько примеров нужно

Цель	Ориентир
Формат и стиль ответа	Сотни качественных примеров
Узкая классификация или извлечение	Тысячи примеров
Новая предметная область	Десятки тысяч и больше
Новый язык	Продолжение предобучения на текстах

- Ориентиры зависят от модели и задачи; их проверяют кривой качества по объёму данных.
- Разнообразие примеров важнее повторения похожих формулировок.

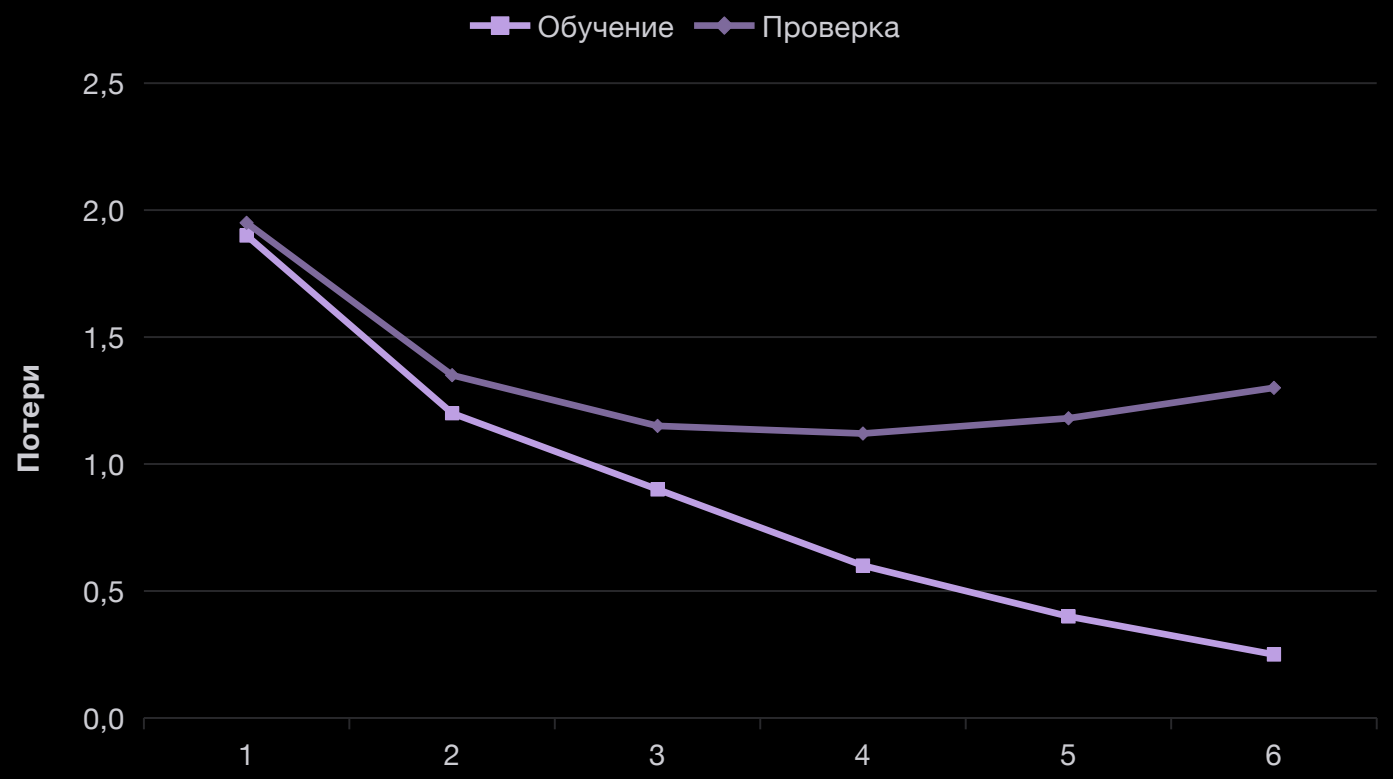
Качество данных важнее объёма

- В работе LIMA около тысячи тщательно отобранных примеров дали заметное улучшение диалогового поведения.
- Ошибки в эталонах модель воспроизводит так же уверенно, как правильные ответы.
- Противоречивые примеры для похожих входов размывают правило, которое должна выучить модель.
- Небольшая ручная проверка выборки часто полезнее, чем удвоение числа примеров.

Переобучение и забывание

- При долгом обучении на малом наборе модель запоминает примеры и хуже обобщает.
- Катастрофическое забывание — ухудшение умений, не представленных в новых данных.
- Признак: модель хорошо решает новую задачу, но перестаёт следовать общим инструкциям.
- Помогают меньшее число эпох, малая скорость обучения, адаптеры и смешивание с общими данными.

Потери на обучении и проверке



- Потери на обучении продолжают снижаться, на проверке начинают расти.
- Расхождение — признак запоминания: выбирают контрольную точку до роста.
- Значения условные и иллюстрируют форму кривых.

Формат данных инструкций

- Каждый пример явно разделяет задание, входные данные и ожидаемый ответ.
- Для диалоговых моделей пример хранится как список сообщений с ролями.
- Формат хранения и шаблон подачи в модель — разные уровни: первое преобразуют во второе.
- Единый формат упрощает проверку, разбиение выборки и повторное использование данных.

Формат Alpaca

```
{
  "instruction": "Определи категорию обращения.",
  "input": "После отмены заказа деньги не вернулись.",
  "output": "Возврат"
}
```

- Формат предложен для набора Stanford Alpaca: одна инструкция и один ответ.
- Поле input может быть пустым, если задание не требует дополнительного текста.
- Многоходовые диалоги в этом формате не представлены.

Формат ShareGPT и Vicuna

```
{"conversations": [
  {"from": "human", "value": "Где мой заказ?"},
  {"from": "gpt",   "value": "Назовите номер заказа."},
  {"from": "human", "value": "12345"},
  {"from": "gpt",   "value": "Заказ передан курьеру."}
]}
```

- Использовался для обучения Vicuna на диалогах; поддерживает несколько ходов.
- Обычно обучают на всех репликах ассистента, а реплики человека маскируют.

Формат сообщений

```
{"messages": [
  {"role": "system",    "content": "Отвечай кратко."},
  {"role": "user",      "content": "Где мой заказ?"},
  {"role": "assistant", "content": "Назовите номер заказа."}
]}
```

- Совпадает с форматом запросов к диалоговым API, знакомым по прошлому семинару.
- TRL принимает такой столбец напрямую и сам применяет шаблон модели.

Сравнение форматов

Формат	Диалоги	Системная роль	Типичное применение
Alpaca	Нет	Нет	Одношаговые задания
ShareGPT / Vicuna	Да	Иногда	Многоходовые диалоги
Сообщения с ролями	Да	Да	Современные наборы и TRL

- Перевод между форматами механический; важно не потерять роли и границы сообщений.

Шаблон диалога

- Шаблон превращает список сообщений в одну строку со служебными токенами конкретной модели.
- Модель обучалась на своём шаблоне; другой шаблон ухудшает ответы даже без ошибок в коде.
- Шаблон хранится вместе с токенизатором и применяется функцией apply_chat_template.
- Добавление начала ответа ассистента при генерации задаётся отдельным параметром.

Шаблон семейства Qwen

```
<|im_start|>system
Отвечай кратко.<|im_end|>
<|im_start|>user
Где мой заказ?<|im_end|>
<|im_start|>assistant
Назовите номер заказа.<|im_end|>
```

- Каждое сообщение ограничено служебными токенами начала и конца.
- Модель учится выдавать <|im_end|> после ответа — это сигнал остановки.

Шаблон семейства Llama 3

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Отвечай кратко.<|eot_id|><|start_header_id|>user<|end_header_id|>

Где мой заказ?<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Назовите номер заказа.<|eot_id|>
```

- Та же структура сообщений, но другие служебные токены и правила пробелов.
- Данные в формате сообщений позволяют сменить модель без переписывания набора.

Применение шаблона в коде

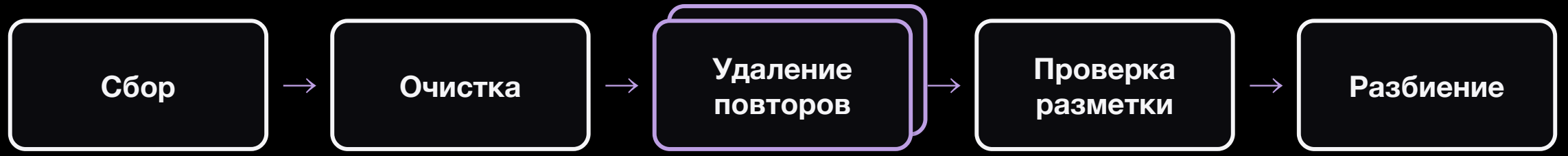
```
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct")
messages = [{"role": "user", "content": "Где мой заказ?"}]
text = tok.apply_chat_template(messages, tokenize=False,
                               add_generation_prompt=True)
```

- add_generation_prompt=True добавляет начало ответа ассистента для генерации.
- При обучении параметр не нужен: ответ уже содержится в примере.

Частые ошибки с шаблонами

- Двойной служебный токен начала: шаблон его уже добавил, а токенизатор добавил ещё раз.
- Обучение базовой модели на шаблоне, токены которого не обучены и не имеют смысла.
- Разные шаблоны при обучении и применении — например, без системного сообщения.
- Перед запуском печатают один пример целиком: строку, токены и маску потерь.

Подготовка набора данных



Точные и почти точные повторы

- Повторы удаляют до разбиения, иначе копии попадут в обучение и проверку одновременно.
- Проверочную и тестовую части фиксируют до любых экспериментов.
- На каждом шаге сохраняют число примеров и причины исключения.

Источники примеров

Источник	Главный риск
Журналы реальных обращений	Персональные данные и случайные ошибки
Ручная разметка	Стоимость и расхождение разметчиков
Открытые наборы	Лицензия и несоответствие задаче
Генерация большой моделью	Однообразие и ошибки учителя

- Лицензию проверяют для данных и для модели, которая генерировала примеры.

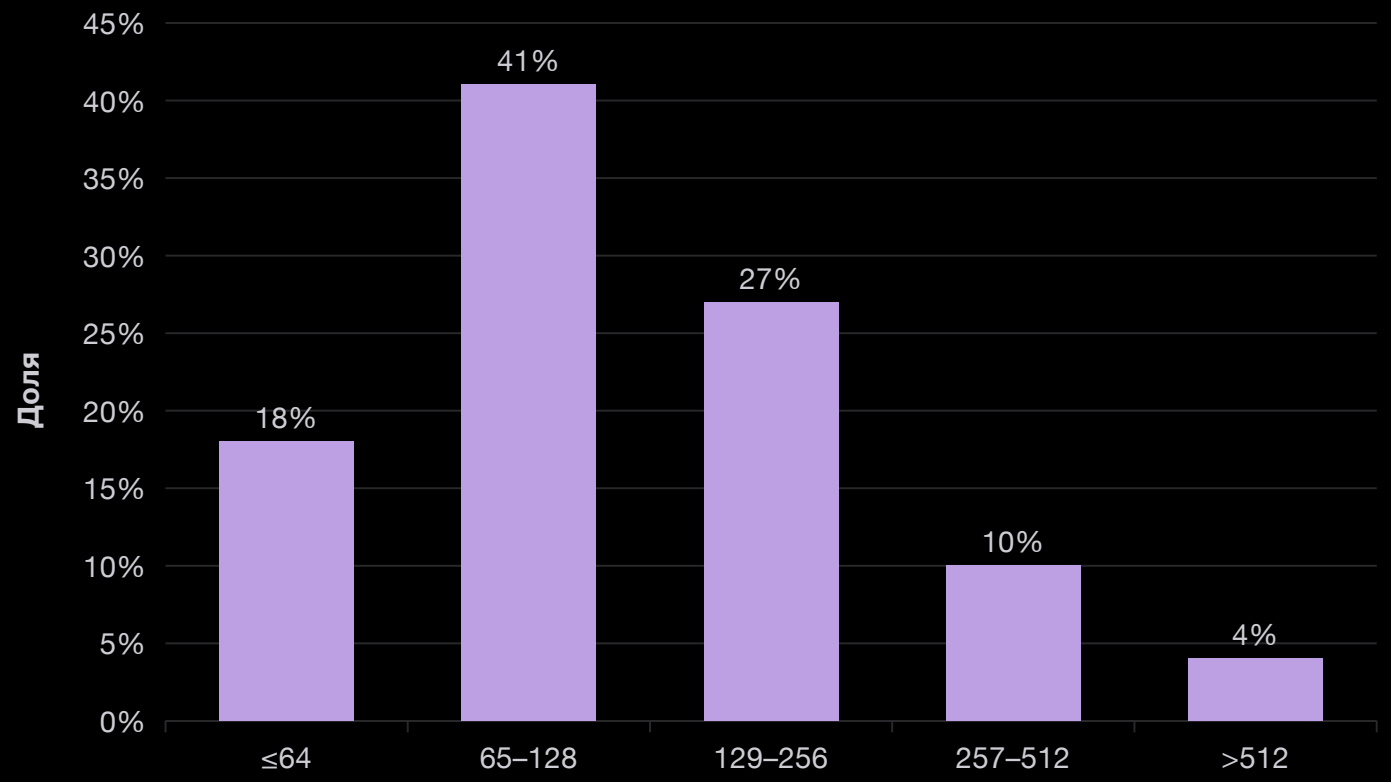
Синтетические примеры

- Большая модель генерирует инструкции и ответы, а небольшая обучается на них.
- Метод быстро даёт объём, но переносит ошибки, стиль и ограничения модели-учителя.
- Нужны фильтры: проверка формата, удаление повторов, выборочная ручная оценка.
- Тестовую выборку собирают независимо, иначе проверка оценивает сходство с учителем.

Проверка набора перед обучением

- Распределение классов, длины входов и ответов, доля пустых и слишком длинных примеров.
- Пересечения между обучением и проверкой — по точному совпадению и по близости текста.
- Соответствие ответов инструкции: нужный язык, формат и допустимые значения.
- Несколько десятков примеров читают вручную до запуска обучения.

Длина примеров в токенах



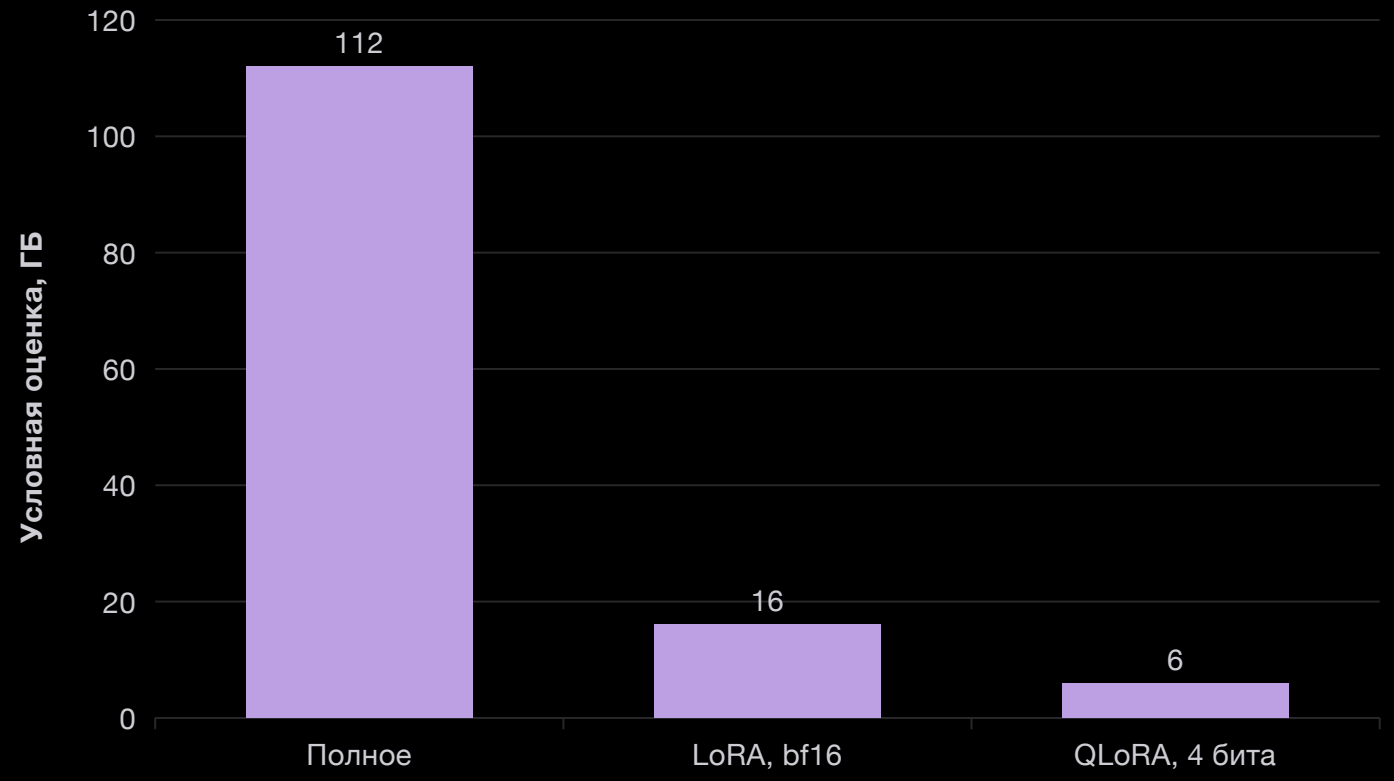
- Максимальная длина задаёт, какие примеры будут обрезаны.
- Обрезанный ответ учит модель обрывать мысль на середине.
- Распределение условное; на семинаре посчитаем его для своего набора.

Память при полном дообучении

Компонент	Байт на параметр
Веса в bf16	2
Градиенты в bf16	2
Основная копия весов в fp32	4
Состояния Adam: два момента в fp32	8
Итого без активаций	≈ 16

- Для модели на 7 млрд параметров это около 112 ГБ без учёта активаций и буферов.
- Схема соответствует обучению со смешанной точностью и оптимизатором Adam.

Память для модели на 7 млрд параметров



- Оценка учитывает веса, градиенты и состояния оптимизатора.
- Активации зависят от длины последовательности и размера пакета.
- Порядок величин важнее точных чисел: они меняются с настройками.

Идея эффективного дообучения

- Основные веса модели замораживаются; обучается небольшая дополнительная часть параметров.
- Градиенты и состояния оптимизатора нужны только для обучаемой части.
- Результат хранится как небольшой адаптер — мегабайты вместо гигабайтов.
- Общее название подхода — parameter-efficient fine-tuning, сокращённо PEFT.

Методы эффективного дообучения

Метод	Что обучается
Адаптеры	Небольшие слои внутри каждого блока
Prompt и prefix tuning	Обучаемые векторы, добавленные ко входу
LoRA	Низкоранговая добавка к матрицам весов
IA^3	Масштабирующие векторы для активаций

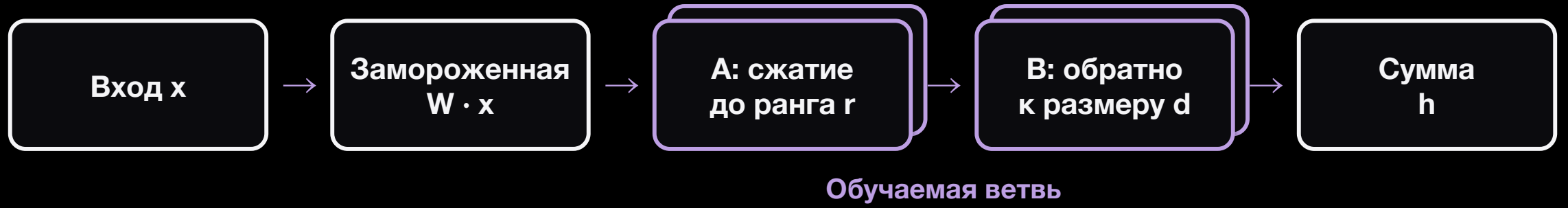
- Библиотека PEFT реализует эти методы с общим интерфейсом.
- На практике чаще всего используют LoRA и её варианты.

Низкоранговая адаптация

$$W' = W + \Delta W = W + (\alpha / r) \cdot B \cdot A$$

- W размера d×k заморожена; обучаются A размера r×k и B размера d×r.
- Ранг r много меньше d и k: изменение веса описывается малым числом параметров.
- B инициализируют нулями, поэтому в начале обучения модель совпадает с исходной.
- Коэффициент α/r масштабирует добавку и связан со скоростью обучения.

Вычисление в слое с LoRA



- Две ветви считаются параллельно и складываются на выходе слоя.
- Градиенты проходят через W, но обновляются только A и B.
- После обучения добавку можно влить в W, и вычисления станут прежними.

Число обучаемых параметров в одной матрице

Вариант	Формула	Параметров при d = k = 4096
Полное обновление	$d \cdot k$	16 777 216
LoRA, r = 8	$r \cdot (d + k)$	65 536
LoRA, r = 16	$r \cdot (d + k)$	131 072
LoRA, r = 64	$r \cdot (d + k)$	524 288

- При r = 16 обучается меньше 1% параметров матрицы.
- Итог по модели зависит от числа слоёв и выбранных матриц.

Гиперпараметры LoRA

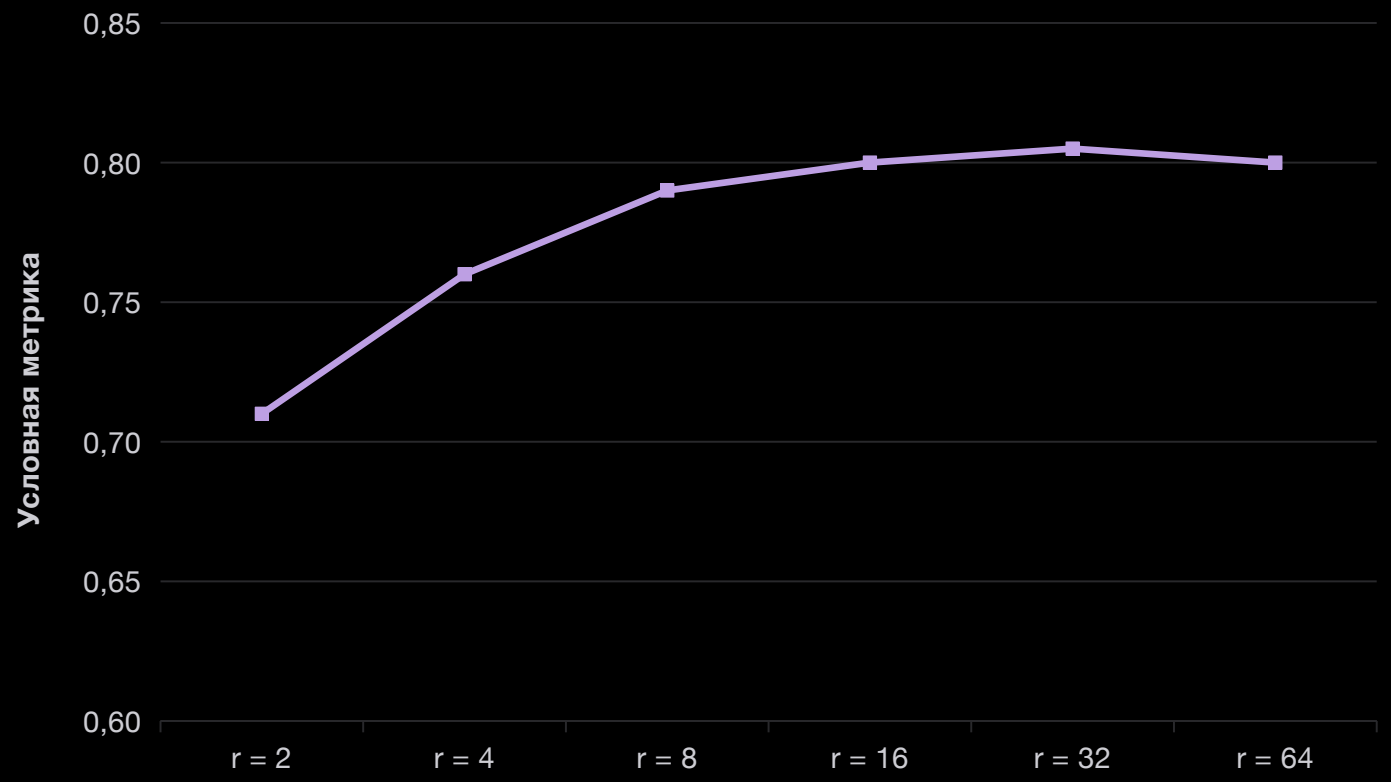
Параметр	Смысл	Частое начальное значение
r	Ранг добавки	8–16
lora_alpha	Масштаб добавки	16–32, часто 2·r
lora_dropout	Регуляризация ветви	0,05
target_modules	Какие матрицы адаптировать	Все линейные слои

- Начальные значения — разумная отправная точка, а не оптимум для любой задачи.

Куда добавлять адаптеры

- В исходной работе LoRA добавляли к проекциям внимания: q_proj и v_proj.
- Позднее показали, что адаптация всех линейных слоёв, включая MLP, обычно даёт лучшее качество.
- Больше целевых матриц — больше обучаемых параметров и памяти, но меньше зависимость от ранга.
- Названия слоёв различаются между семействами: их смотрят в описании модели.

Ранг и качество



- Рост ранга после некоторого значения мало меняет результат.
- Для узкой задачи часто достаточно небольшого ранга.
- Кривая условная: её форму проверяют на своей задаче.

Слияние и хранение адаптеров

- Адаптер хранится отдельно от базовой модели и весит единицы или десятки мегабайт.
- Для применения добавку можно влить в веса: merge_and_unload возвращает обычную модель.
- Влитая модель не требует дополнительных вычислений, но занимает место полной копии.
- Адаптер работает только с той базовой моделью и ревизией, на которой он обучен.

Несколько адаптеров на одной модели

- Одна загруженная базовая модель может обслуживать несколько задач с разными адаптерами.
- Адаптер выбирают для каждого запроса; переключение не требует перезагрузки весов.
- Серверы инференса, например vLLM, поддерживают такой режим.
- Это удобно для разных клиентов или языков при общей исходной модели.

QLoRA

- Базовая модель хранится в 4-битном формате NF4; адаптеры LoRA обучаются в 16 битах.
- Двойная квантизация сжимает и сами коэффициенты масштаба, экономя ещё немного памяти.
- Страничные оптимизаторы переносят состояния в оперативную память при пиках нагрузки.
- В работе QLoRA модель на 65 млрд параметров дообучали на одном GPU с 48 ГБ.

Хранение весов базовой модели

Формат	Байт на параметр	Модель на 7 млрд
fp32	4	≈ 28 ГБ
bf16	2	≈ 14 ГБ
int8	1	≈ 7 ГБ
NF4	≈ 0,5	≈ 3,5–4 ГБ

- Квантизация экономит память, но добавляет вычисления на распаковку весов.
- Подробнее о форматах — в блоке о развёртывании моделей.

Ограничения QLoRA

- Обучение медленнее, чем LoRA в bf16, из-за распаковки весов на каждом шаге.
- bitsandbytes ориентирован на GPU NVIDIA; на других устройствах поддержка ограничена.
- Адаптер обучен поверх квантованной модели; при слиянии с bf16-весами возможны расхождения.
- Качество после обучения проверяют в том же формате, в котором модель будет работать.

Развитие идеи LoRA

- DoRA разделяет обновление веса на изменение длины и направления.
- rsLoRA меняет масштаб на $\alpha/\sqrt{r}$, чтобы большие ранги обучались устойчивее.
- LoRA+ использует разные скорости обучения для матриц A и B.
- Варианты включаются параметрами LoraConfig; выигрыш проверяют на своей задаче.

Инструменты Hugging Face

Transformers	Модели, токенизаторы, шаблоны диалога
Datasets	Загрузка, преобразование и разбиение данных
PEFT	LoRA, QLoRA и другие адаптеры
TRL	Циклы SFT, DPO и обучения с вознаграждением
Accelerate	Устройства, смешанная точность, распределение

- TRL построена поверх Trainer из Transformers и принимает конфигурацию PEFT.

Конфигурация LoRA в PEFT

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                    target_modules="all-linear", task_type="CAUSAL_LM")
model = get_peft_model(model, config)
model.print_trainable_parameters()
```

- print_trainable_parameters показывает долю обучаемых параметров.
- target_modules="all-linear" выбирает все линейные слои, кроме выходного.

Обучение в TRL

```
from trl import SFTConfig, SFTTrainer
args = SFTConfig(output_dir="out", num_train_epochs=1,
                 learning_rate=2e-4, per_device_train_batch_size=8,
                 assistant_only_loss=True, max_length=512)
trainer = SFTTrainer(model=model_id, args=args, train_dataset=train,
                     eval_dataset=dev, peft_config=config)
trainer.train()
```

- SFTTrainer применяет шаблон диалога к столбцу messages.
- assistant_only_loss оставляет в потерях только ответы ассистента.

Основные параметры обучения

Параметр	Что задаёт	Ориентир для LoRA
learning_rate	Шаг обновления	1e-4 – 2e-4
num_train_epochs	Проходы по данным	1–3
per_device_train_batch_size	Примеров за шаг	По памяти
gradient_accumulation_steps	Накопление шагов	Для нужного пакета
max_length	Длина примера	По распределению

- Скорость обучения для LoRA обычно выше, чем для полного дообучения.

Эффективный размер пакета

$B = \text{batch} \times \text{accumulation} \times \text{число устройств}$

- Накопление градиентов позволяет получить большой пакет при малой памяти.
- Число шагов оптимизатора = число примеров × эпохи / B.
- При изменении B часто пересматривают и скорость обучения.
- Для воспроизводимости сохраняют все три множителя, а не только итог.

Скорость обучения и расписание

- Разогрев: скорость постепенно растёт в начале, чтобы избежать резких обновлений.
- Затем скорость снижается — линейно или по косинусу — к концу обучения.
- Слишком большая скорость даёт скачки потерь, слишком малая — медленное обучение.
- Короткий пробный запуск на части данных помогает выбрать порядок величины.

Экономия памяти и времени

- Gradient checkpointing пересчитывает активации на обратном проходе вместо хранения.
- Упаковка коротких примеров в одну последовательность уменьшает долю выравнивания.
- Смешанная точность bf16 сокращает память и ускоряет вычисления на поддерживающих устройствах.
- Каждый приём меняет скорость или память, но не должен менять результат по качеству.

Оценка до и после дообучения

- Исходную модель оценивают на той же проверке, с тем же шаблоном и параметрами генерации.
- Кроме целевой метрики проверяют формат: долю корректных ответов и причины остановки.
- Отдельный набор общих вопросов показывает, не ухудшились ли другие умения.
- Несколько ответов до и после читают вручную: метрика не видит всех изменений.

Перед запуском дообучения

Вопрос	Если ответ «нет»
Есть базовая линия на промптах?	Сначала получить её
Тестовая выборка зафиксирована?	Отложить и не трогать
Пример печатается после шаблона?	Проверить токены и маску
Есть проверка общих умений?	Добавить небольшой набор
Определена метрика успеха?	Сформулировать до обучения

Контрольные вопросы

- Чем дообучение отличается от добавления примеров в запрос?
- Почему токены инструкции обычно исключают из функции потерь?
- Что произойдёт, если обучать модель без токена завершения ответа?
- Почему LoRA требует меньше памяти, хотя прямой проход идёт через все веса?
- Какие проверки нужны, чтобы заметить забывание после дообучения?

Материалы и подготовка к семинару

- На семинаре: подготовим набор инструкций, дообучим небольшую модель с LoRA и сравним её с исходной.

LoRA: Low-Rank Adaptation of Large Language Models

QLoRA: Efficient Finetuning of Quantized LLMs

LIMA: Less Is More for Alignment

Документация PEFT

Документация TRL: SFTTrainer